

How KV-Caches Make LLM Inference Fast

Introduction to MLOps

Mahdi Abbasi

Probationary lecture · AI Infrastructure · THWS Würzburg · 6 October 2026

Code & notebook: github.com/Mahdi-abbasi/358/KV-Cache



scan me

Why inference?

A model is **trained** once.

It is **used** millions of times.

Why does the first word take longer than the rest?

What do we measure?

TTFT

time to first token: how long the user waits

TPOT

time per output token: how fast the text streams

Throughput

tokens per second per GPU: what it costs

Cache memory

Affects the performance as we'll see

Today's plan

- 1 **Text generation** — how GPT-2 makes one token
- 2 **Many tokens** — why the simple loop gets slow
- 3 **KV caching** — prefill, decode and a live demo

Next session: serving at scale — cache memory, vLLM, batching

Text generation

Our model: GPT-2

- ▶ open, small, fast
- ▶ 124M parameters
- ▶ 12 layers, hidden size 768
- ▶ vocabulary: 50,257 tokens
- ▶ e.g. **autocomplete**: answers in milliseconds

Loading GPT-2

```
from transformers import AutoModelForCausalLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")
print(model)
```

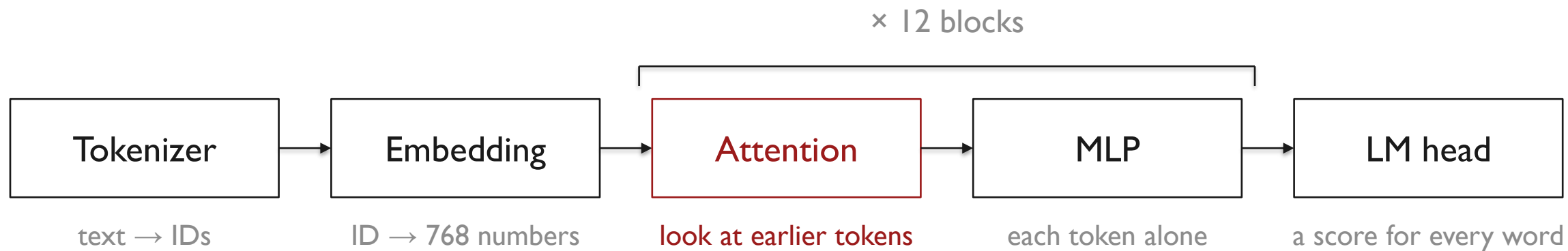
Loading GPT-2

```
GPT2LMHeadModel(  
  (transformer): GPT2Model(  
    (wte): Embedding(50257, 768)  
    (wpe): Embedding(1024, 768)  
    (h): 12 x GPT2Block(  
      (attn): GPT2Attention(  
        (c_attn): Conv1D(nf=2304, nx=768)  
        (c_proj): Conv1D(nf=768, nx=768))  
      (mlp): GPT2MLP(  
        (c_fc): Conv1D(nf=3072, nx=768)  
        (c_proj): Conv1D(nf=768, nx=3072)))  
    (lm_head): Linear(768 -> 50257)
```

$2304 = 3 \times 768$: Q, K, V

K and V are what we will cache.

Inside GPT-2



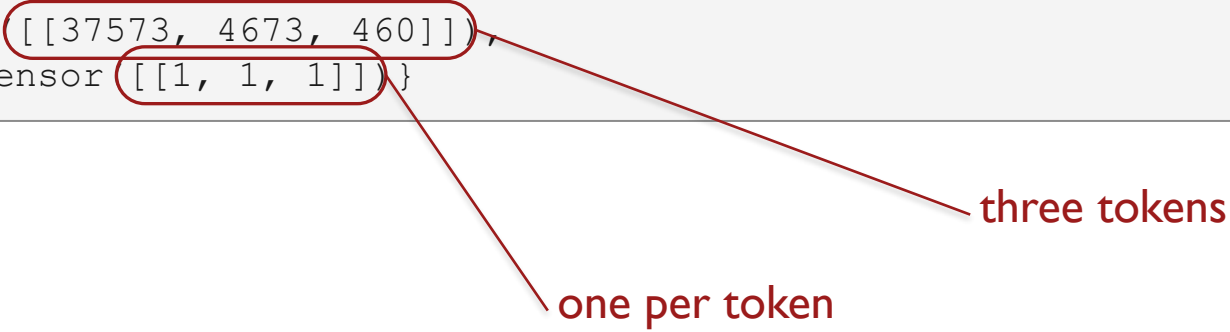
Attention makes **Q**, **K**, **V** for every token.

K and **V** are what the KV cache stores.

Tokenize the prompt

```
inputs = tokenizer("Machine learning can", return_tensors="pt")
```

```
{'input_ids': tensor([[37573, 4673, 460]]),  
 'attention_mask': tensor([[1, 1, 1]])}
```



three tokens

one per token

['Machine', ' learning', ' can'] ← the space is part of the token

One forward pass

```
with torch.no_grad():  
    logits = model(**inputs).logits  
    print(logits.shape)
```

```
torch.Size([1, 3, 50257])
```

batch

tokens

vocabulary

Pick the next token

```
last_logits = logits[0, -1, :]  
next_token_id = last_logits.argmax()  
tokenizer.decode(next_token_id)
```

last position

' be' (token 307)

Machine learning can **be**

Greedy: take the best

Append and repeat

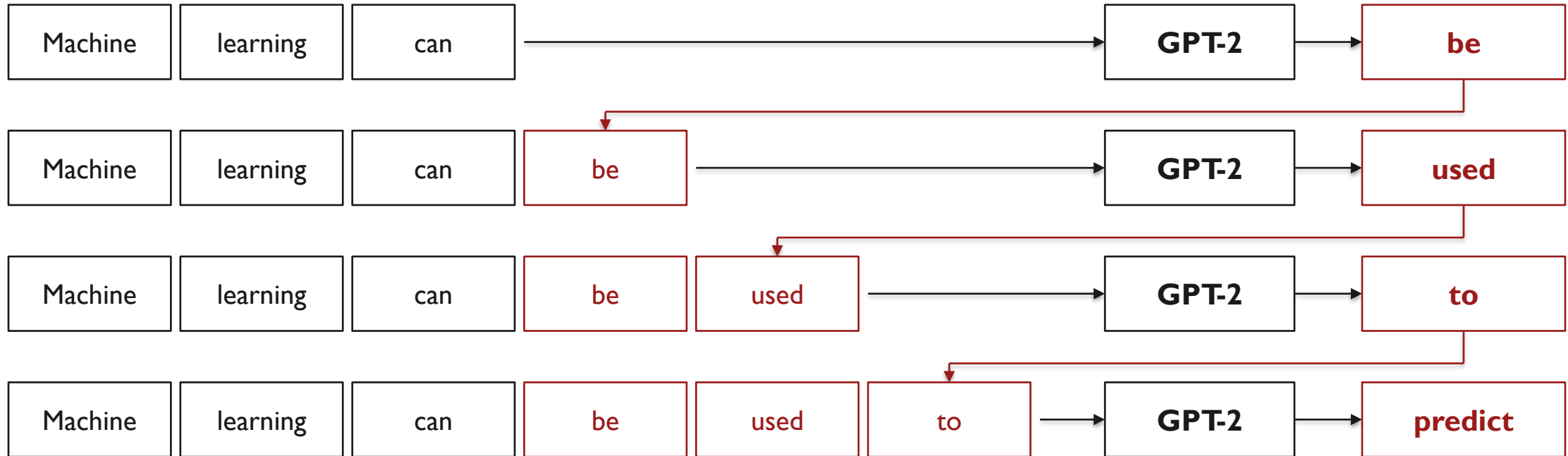
```
input_ids = torch.cat([input_ids, next_token_id.reshape(1, 1)], dim=1)
```

```
tensor([[37573, 4673, 460, 307]])  
torch.Size([1, 4])
```

the new token

The **whole sequence** goes in again.

Autoregressive: one token at a time



Each new token goes back into the input. **The model reads everything again.**

Generating many tokens

A helper: generate_token

```
def generate_token(inputs):  
    with torch.no_grad():  
        outputs = model(**inputs)  
    return outputs.logits[0, -1, :].argmax()
```

inference: no gradients

one forward pass

greedy: best token

last position

The same steps as before (forward pass → last position → best token)

packed in one function: **input tokens in, next token ID out.**

Next: we call it 50 times.

The simple loop

```
next_inputs = inputs
for _ in range(50):
    t0 = time.perf_counter()
    next_token_id = generate_token(next_inputs)
    durations.append(time.perf_counter() - t0)
    ids, mask = next_inputs["input_ids"], next_inputs["attention_mask"]
    next_inputs = {
        "input_ids": torch.cat([ids, next_token_id.reshape(1, 1)], dim=1),
        "attention_mask": torch.cat([mask, torch.tensor([[1]]), dim=1)]
```

start: the tokenized prompt (slide 10)

add the new token at the end

...and a 1 in the mask

What goes into `generate_token` in each step

step	next_inputs["input_ids"]	attention_mask	tokens	output
1	Machine learning can	[1, 1, 1]	3	" be"
2	Machine learning can be	[1, 1, 1, 1]	4	" used"
3	Machine learning can be used	[1, 1, 1, 1, 1]	5	" to"
...	...			
50	prompt + 49 new tokens	52 ones	52	

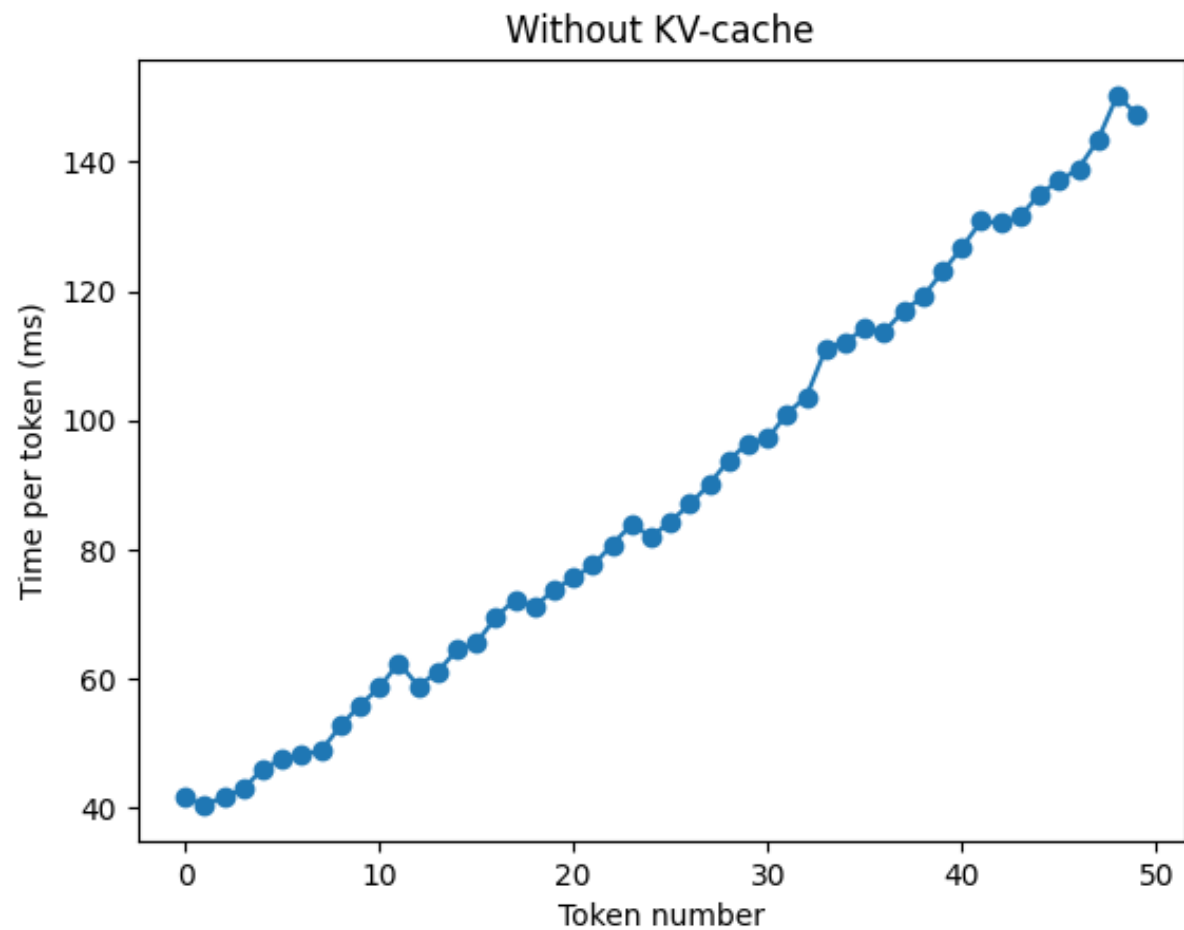
input_ids hold token IDs (slide 10); here we show them as words. The input grows by one token every step.

The simple loop

```
next_inputs = inputs
for _ in range(50):
    t0 = time.perf_counter()
    next_token_id = generate_token(next_inputs)
    durations.append(time.perf_counter() - t0)
    ids, mask = next_inputs["input_ids"], next_inputs["attention_mask"]
    next_inputs = {
        "input_ids": torch.cat([ids, next_token_id.reshape(1, 1)], dim=1),
        "attention_mask": torch.cat([mask, torch.tensor([[1]])], dim=1)}
```

Time per token: flat, or rising?

Result: rising



4.5 s for 50 tokens

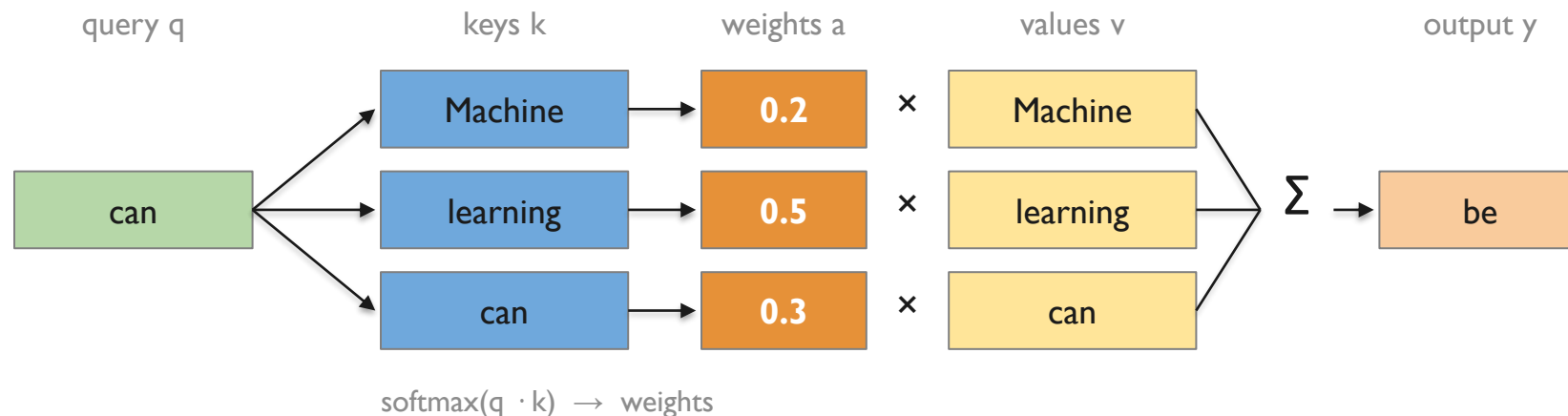
40 → 150 ms per token

KV caching

Q, K, V: three roles for every token

Query q	Key k	Value v
What am I looking for? The token that is being processed asks. library: search text	What do I contain? Every token shows a label to be matched. library: book titles	What do I give? Every token passes on its content. library: the books

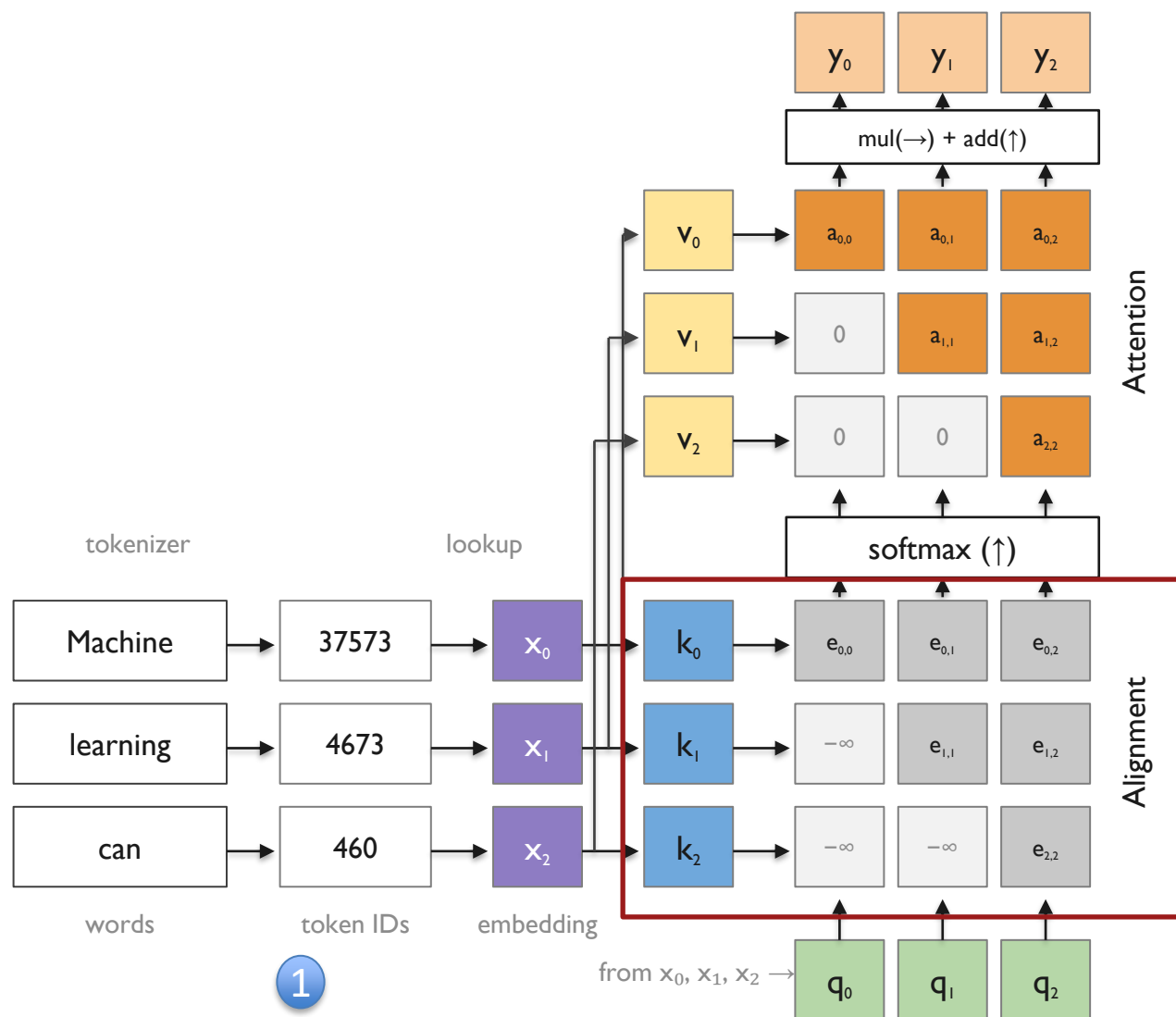
Example: the token “can” looks at the prompt



- ▶ “can” needs only **its own q**.
- ▶ But it needs k and v of **all tokens so far**.

Weights 0.2 / 0.5 / 0.3 are only an example. $a = \text{softmax}(q \cdot k / \sqrt{d})$, $y = \sum a \cdot v$

Causal attention: the burden of K and V



Inputs:

- words $\rightarrow$ token IDs $\rightarrow$ embeddings x

Operations:

- Query vectors: $q = W_q^T x$
- Key vectors: $k = W_k^T x$
- Value vectors: $v = W_v^T x$
- Alignment:** $e_{i,j} = q_j \cdot k_i / \sqrt{D_k}$
- Attention:** $a = \text{softmax}(e + \text{mask})$
- Output: $y_j = \sum_i a_{i,j} v_i$

Outputs: context vectors y (D_v)

Autoregressive + causal: every new token needs K and V of all previous tokens.

The causal mask

token that looks (queries) →

	Machine	learning	can	be
Machine				
learning	$-\infty$			
can	$-\infty$	$-\infty$		
be	$-\infty$	$-\infty$	$-\infty$	

↑ keys

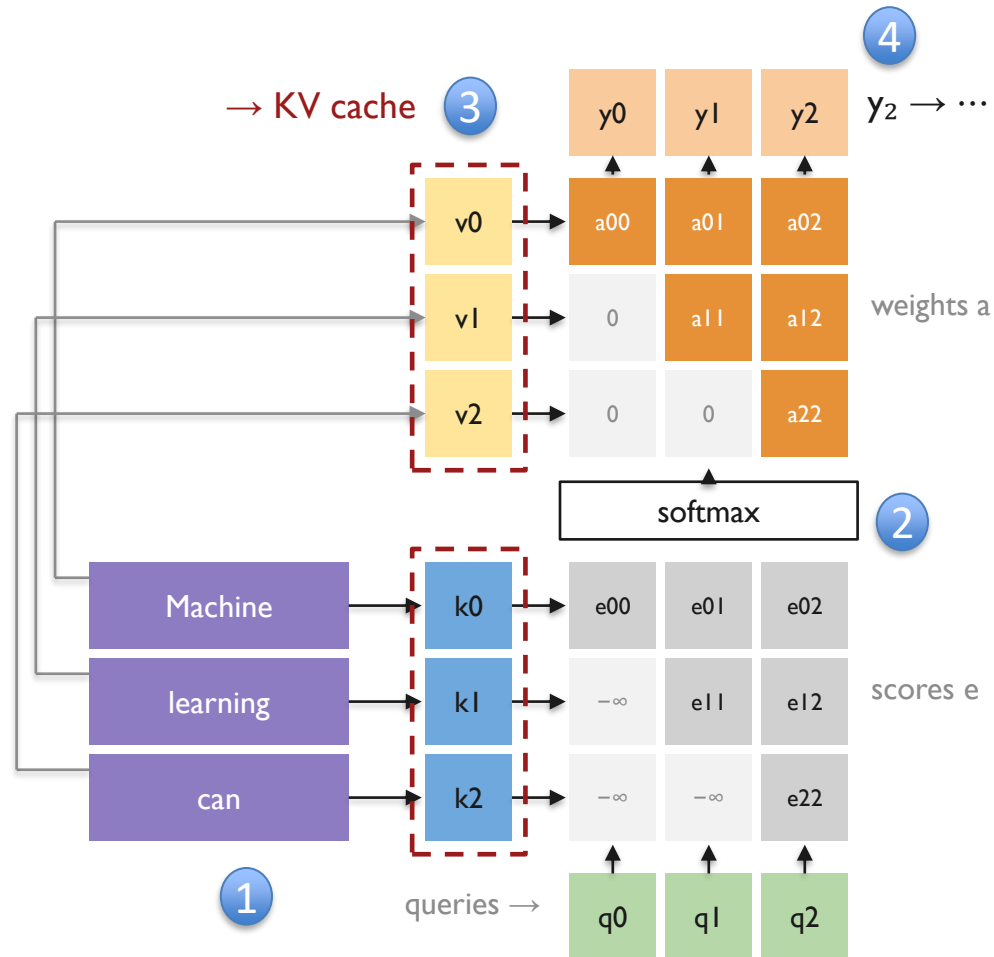
red: allowed, the past

$-\infty$: the future is hidden

after softmax: weight 0

rows = keys · columns = queries

Prefill: attention → “be”

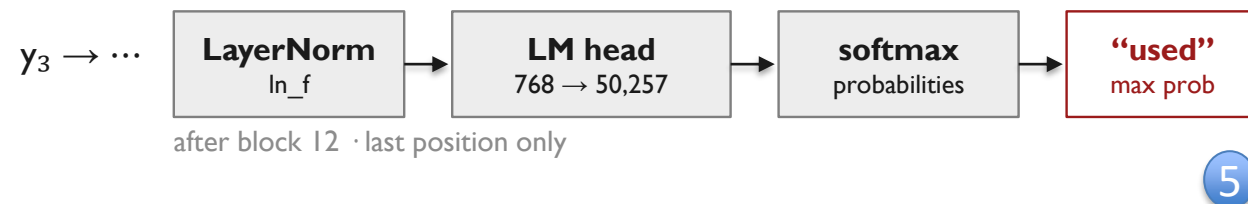
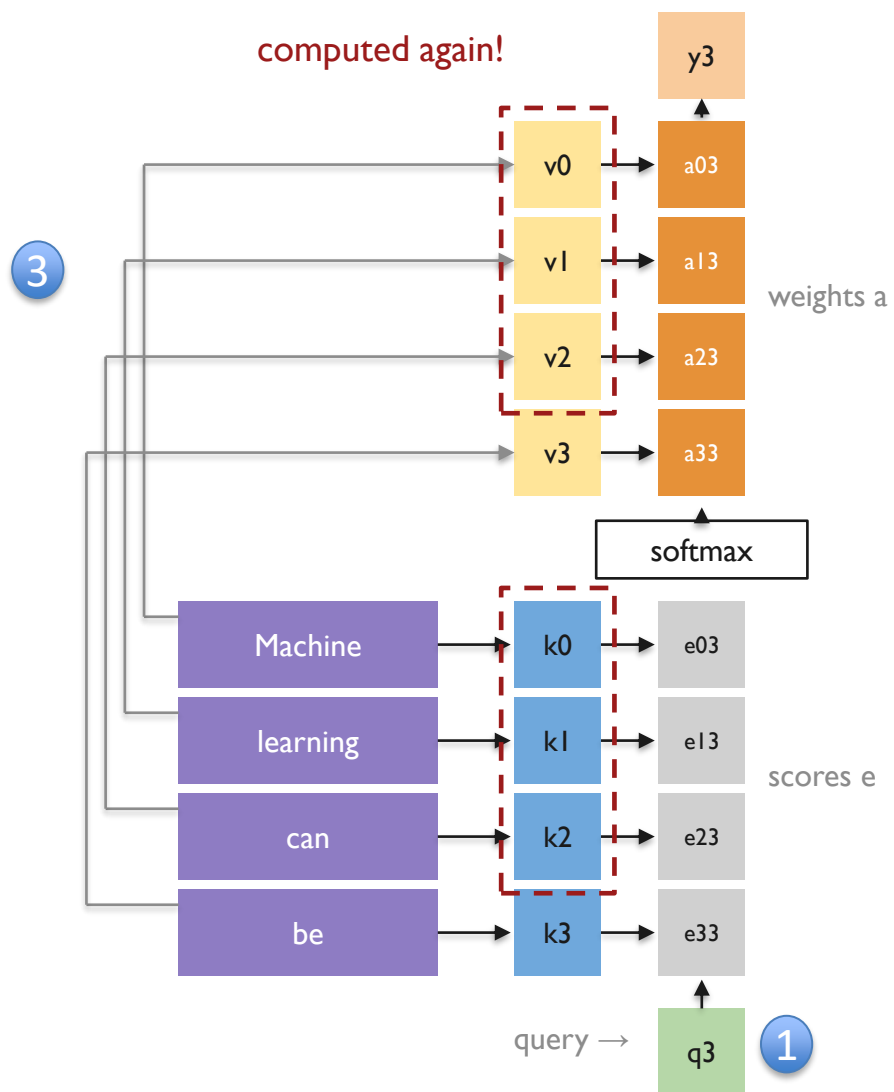


KV cache (per layer)

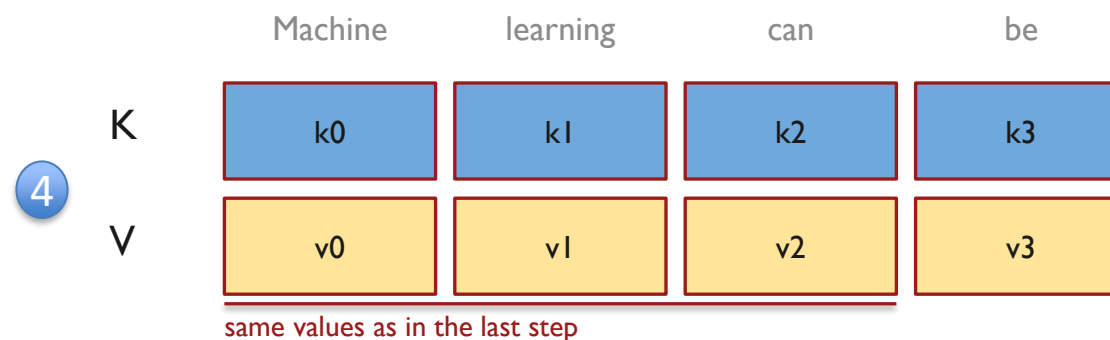
	Machine	learning	can
K	k0	k1	k2
V	v0	v1	v2

- ▶ all 3 tokens at once
- ▶ $-\infty$: causal mask (future token)
- ▶ **red dashed**: k, v saved to the cache
- ▶ rows = keys · columns = queries

Next step without a cache: everything again



Without a cache, old K and V are recomputed in every step



- 2 ▶ all 4 tokens go in again
- ▶ recompute: k_0-k_2, v_0-v_2 from x
- ▶ needed: only $q_3 \rightarrow$ one column
- ▶ every step: $O(t)$ tokens, 12 layers

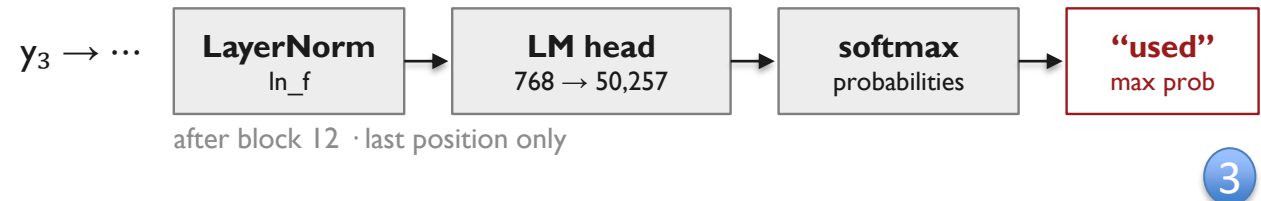
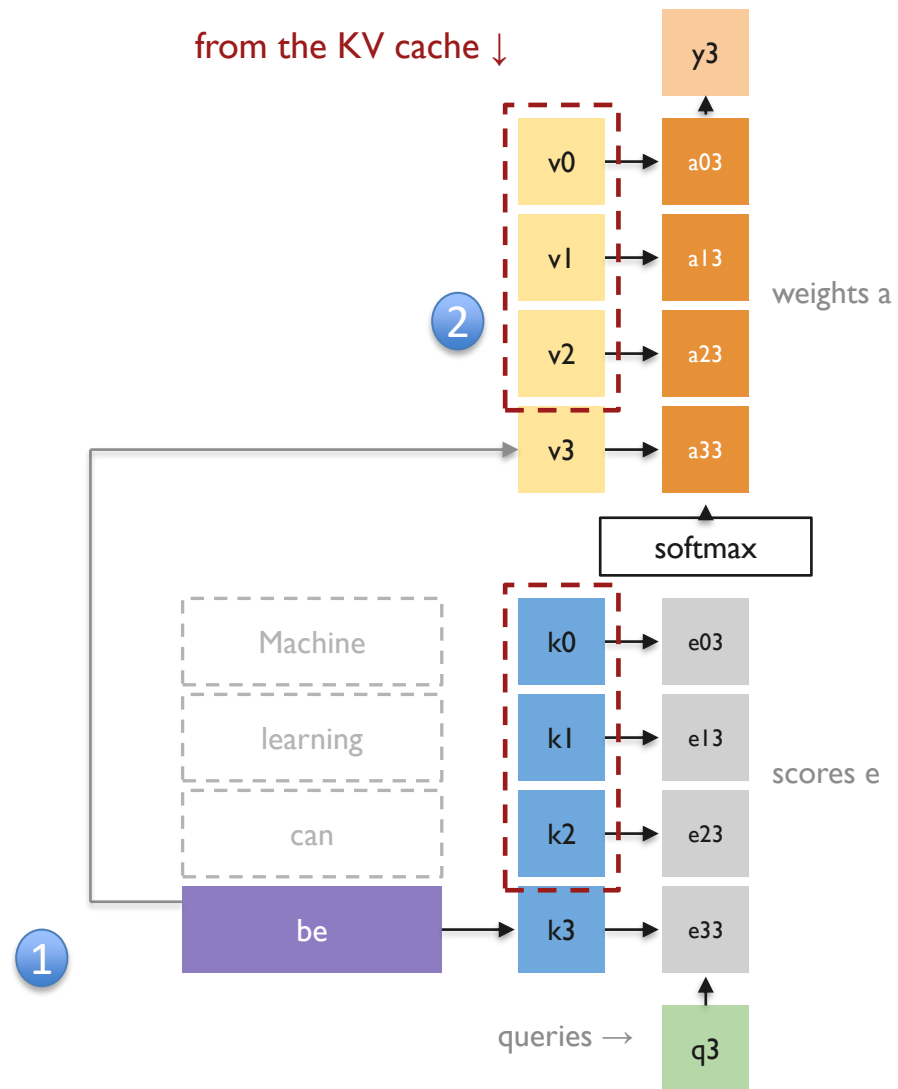
The key insight

Old **keys** and **values** never change.

→ compute once, store, reuse

= the **KV** cache

Next step with the KV cache: only “be”



KV cache (per layer)

	Machine	learning	can	be
K	k0	k1	k2	k3
V	v0	v1	v2	v3

- **read:** k_0 – k_2 , v_0 – v_2 of the prompt
- **compute:** q , k , v only for “be”
- **add:** k_3 , v_3 to the cache
- dashed grey: not recomputed

Two phases, two metrics

Prefill → time to first token (TTFT)

Decode → time per output token (TPOT)

The loop with a cache

```
for _ in range(50):  
    next_token_id, past_key_values = generate_token_with_past(next_inputs)  
    next_inputs = {  
        "input_ids": next_token_id.reshape((1, 1)),  
        "attention_mask": torch.cat([mask, torch.tensor([[1]])], dim=1),  
        "past_key_values": past_key_values
```

only the new token

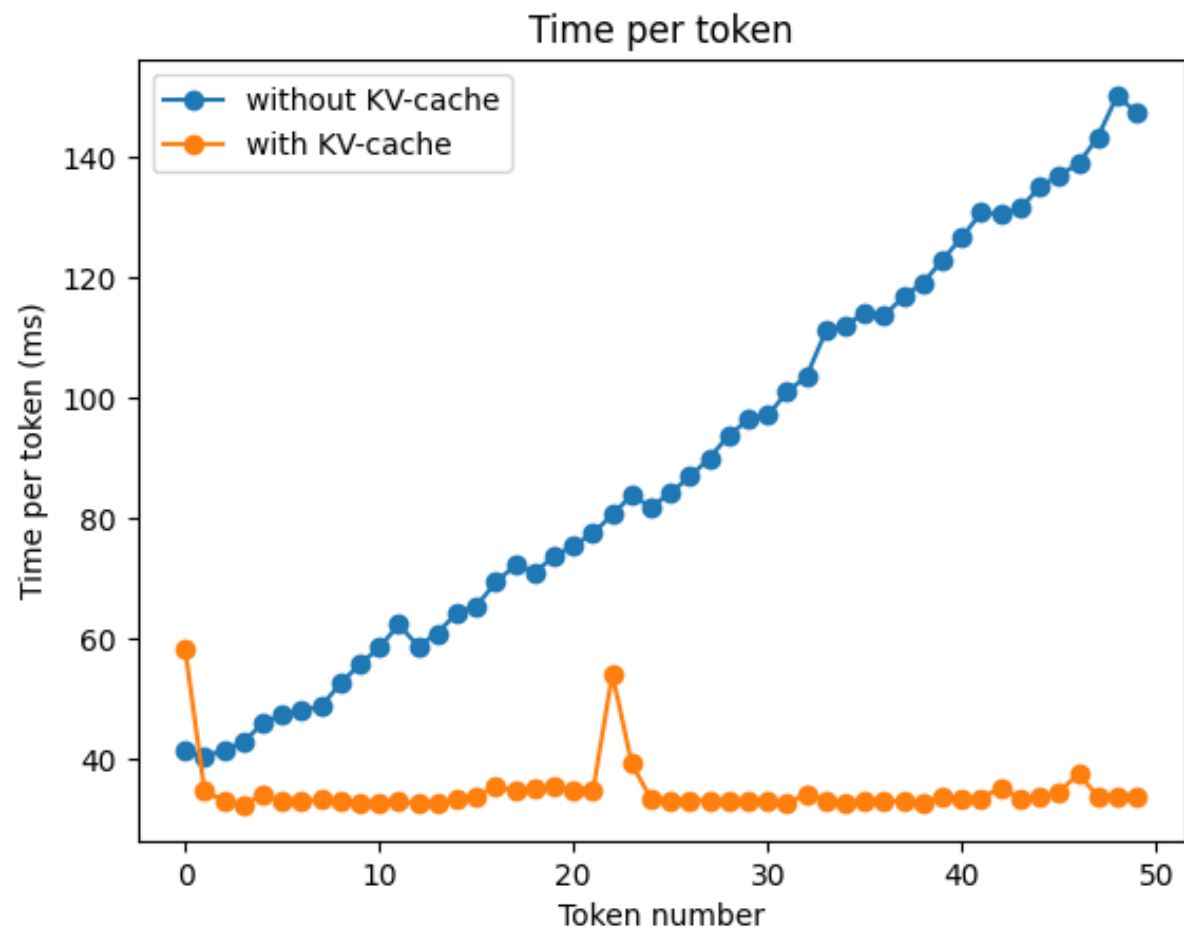
the cache

attention_mask: still the full length

Live demo

Is the first step faster with the cache?

Result: flat



without cache: rising

with cache: flat, ~34 ms

step 1 = prefill

Result in numbers

	without cache	with cache
TTFT	41.5 ms	58.4 ms
TPOT	90.1 ms	34.2 ms
50 tokens	4.46 s	1.74 s

61 % less time · same text

Summary

- ▶ One token per forward pass
- ▶ Old K,V never change → **cache** them
- ▶ **Prefill** once, then **decode**
- ▶ The cache trades compute for memory

Quick check

Your prompt becomes twice as long.

Which grows more: **TTFT** or **TPOT**?

Quick check

Your prompt becomes twice as long.

Which grows more: TTFT or TPOT?

TTFT: the prefill reads the whole prompt.

That is why the first word is slow.

Homework

Use the notebook. Hand in one page with your plots and a short answer to each task.

- 1 Prompt length.** Use prompts with about 10, 100 and 500 tokens. Measure TTFT and TPOT, with and without the cache. Plot them. What grows, and why?
- 2 Memory.** Read the size of `past_key_values` after 50 tokens. Check it with $2 \times \text{layers} \times \text{hidden} \times \text{bytes} \times \text{tokens}$.
- 3 Decoding.** Use `model.generate` with `do_sample=True`, temperature 0.7 and `top_k` 50. Compare with greedy. Does the cache change the text?
- 4 Bonus.** Keep only the last 32 tokens in the cache (a sliding window). What happens to speed and to the text?

Further reading

Foundations

- **Vaswani et al.** Attention Is All You Need. NeurIPS 2017.
- **Radford et al.** Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI 2019.
- **J. Alammar.** The Illustrated GPT-2. jalammar.github.io

KV cache, hands-on

- **S. Raschka.** Understanding and Coding the KV Cache in LLMs from Scratch. Blog, 2025.
- **Hugging Face.** Cache strategies. huggingface.co/docs/transformers/kv_cache
- **DeepLearning.AI.** Efficiently Serving LLMs (short course).

Serving at scale

- **Pope et al.** Efficiently Scaling Transformer Inference. MLSys 2023.
- **Kwon et al.** Efficient Memory Management for LLM Serving with PagedAttention (vLLM). SOSP 2023.
- **Shazeer 2019 (MQA); Ainslie et al. 2023 (GQA).** Fewer K/V heads, smaller cache.

Questions?

Next session: serving at scale → memory, vLLM, batching



Code & notebook: github.com/Mahdi-abbasi/358/KV-Cache

Backup

for questions only

Next session: serving at scale

The price: memory

`2 × layers × hidden × bytes × tokens`

- ▶ **GPT-2**: 72 KB per token
- ▶ **Llama 2 7B**: 512 KB per token → 2 GB for 4,096 tokens

Many users: the **cache**, not the model, fills the GPU.

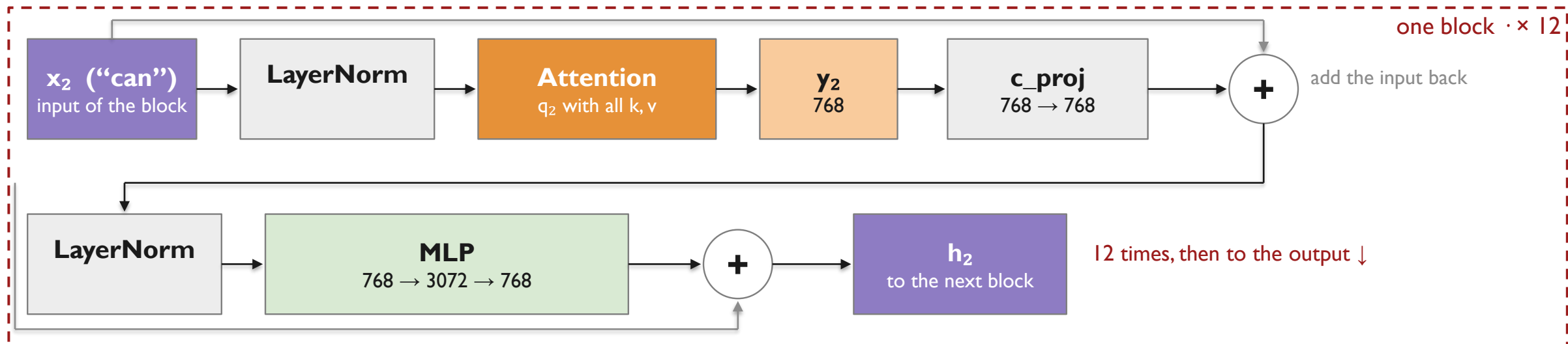
Serving systems

- ▶ **Paged** cache: small blocks, like memory pages (vLLM)
- ▶ **Prefix** caching: share a common system prompt
- ▶ **Fewer K/V heads**: smaller cache by design
- ▶ **8-bit** cache, or drop old tokens

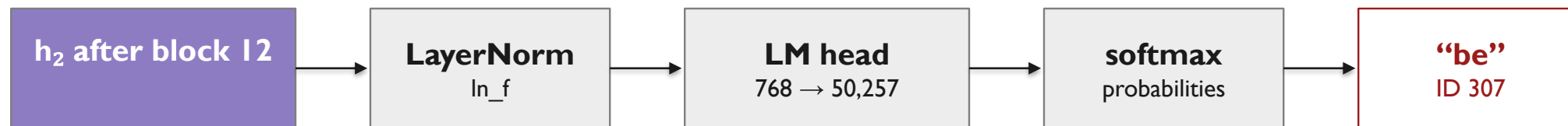
Monitor: **TTFT** · **TPOT** · **cache usage**

After attention: from y_2 to “be”

We follow the last token, “can”, through **one block** and then to the output.



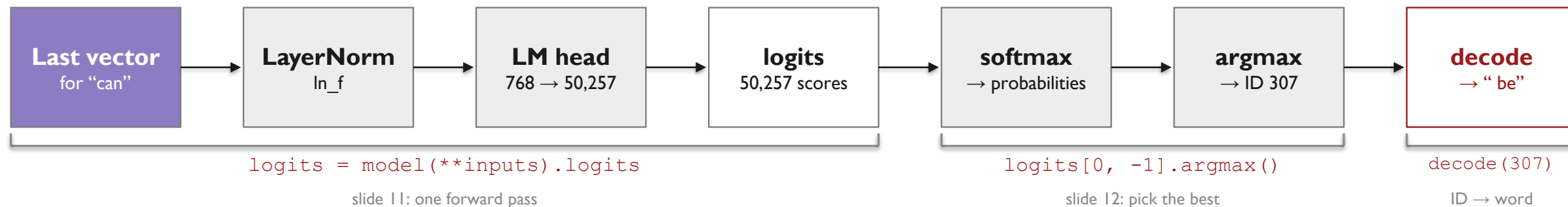
Output (once, after the last block)



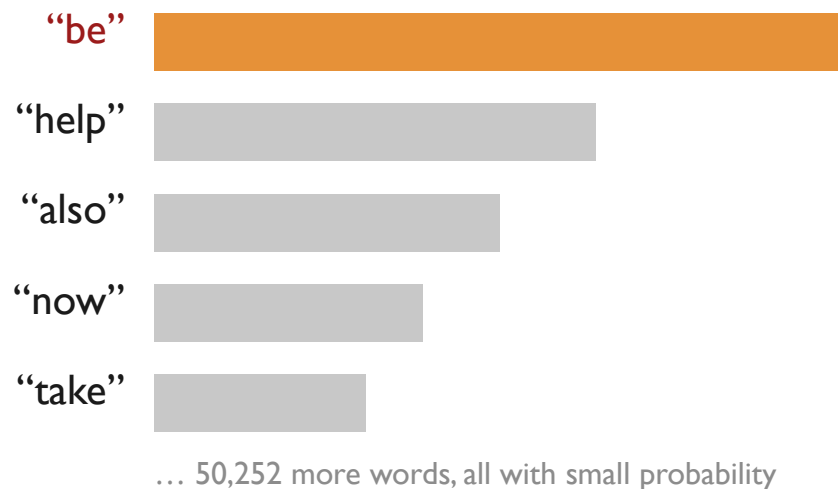
Every token goes through every block. For the next word, only the **last position** goes to the LM head.

From the last vector to “be”

from GPT-2's 12 blocks — what happens inside comes later



Top candidates after “Machine learning can”



- ▶ **LayerNorm**: numbers of the token → similar size (mean 0, std 1)
- ▶ **LM head**: one score for every word in the vocabulary
- ▶ **softmax**: scores → probabilities, sum = 1
- ▶ **greedy**: take the top → “be”
- ▶ **sampling**: draw from the probabilities → sometimes “help”

Order of candidates from our notebook (top-k); bar lengths are illustrative. Real probabilities: notebook, Part 2.